



Exploiting Novel Software Development Paradigms to Increase the Sustainability of Data Centers

A. Carrega
DITEN - University of Genoa, Italy
alessandro.carrega@unige.it

M. Repetto
CNIT – RU of Genoa, Italy
matteo.repetto@cnit.it

ABSTRACT

The application of effective energy management strategies in data centers is often hindered by the substantial conflict between the interests of cloud users and infrastructure owners. As a matter of fact, cloud users require that the service level they are paying for is tightly met, whereas data center owners try to cut down their operational expenses.

In this paper, we propose a novel consolidation algorithm that exploits emerging software development paradigms. Our approach enables cloud users to indicate their willingness to apply energy saving mechanisms to some of their virtual resources, hence giving infrastructure managers the ability to apply more efficient workload consolidation and to switch their hardware to very low-power states. The result is an optimal trade-off between energy consumption and performance.

Keywords

green data centers, energy efficiency, software development paradigms, consolidation algorithms

1. INTRODUCTION

Data center operators are often concerned with inefficient usage of their infrastructure. Workload consolidation (i.e., gathering VMs on the smallest number of servers) has been proposed as an effective mean to make the most efficient usage of active servers and to put idle servers into low-power modes; however, consolidation usually implies migration of virtual resources, and this procedure is likely to impact the QoS/QoE granted by service providers to their clients. This fact may represent a substantial hinder to the adoption of energy efficiency solutions in real installations.

We believe that emerging software development paradigms for cloud applications may provide a substantial improvements in the effectiveness of consolidation strategies, while at the same time providing more QoS/QoE opportunities for cloud users. Indeed, the dynamic and flexible nature of cloud applications motivates the interest in autonomic

software management frameworks that are able to take over deployment and life-cycle operations, including monitoring and scaling aspects, from simple template-based deployment engines (e.g., OpenStack Heat [1], AWS CloudFormation [2]) to complex orchestration solutions over multiple clouds (e.g., ETSI NFV MANO [3], ARCADIA [4], Cloudify [5]).

In our opinion, the usage of templates and meta-data in the software development process could be exploited to also drive energy management in data centers. We already sketched a novel architecture for energy-aware consolidation in data centers, based on additional context information included by developers into their applications [6]. In this paper, we outline a new consolidation algorithm in such framework, which optimizes the energy consumption of both computing and networking equipment in a data center. We also show the effectiveness of our approach through simulations.

The paper is organized as follows. Section 2 reviews the background, motivation, and related work behind our work. Section 3 reviews the conceptual framework we are considering, while Section 4 outlines the consolidation algorithm proposed in this work. Section 5 shows the preliminary results from our simulations. Finally, Section 6 gives our conclusions.

2. BACKGROUND AND MOTIVATION

Recent advances in data center design has resulted in large improvement for the ancillary infrastructure: some big players reported they achieved a Power Usage Effectiveness (PUE) factor of 1.08¹; this basically means that IT equipment is now the main responsible for power consumption, hence it is important to focus on optimizing the way resources are allocated and utilized to serve dynamic application workloads.

“Consolidation” of Virtual Machines (VMs) is one of the most effective way to save energy in data centers. Unfortunately, consolidation usually entails frequent migration of VMs, and this may lead to service degradation. We argue that applications running in a data center may have different QoS requirements, making them more or less sensible to side effects brought by migration. In this respect, emerging software development paradigms could provide an effective solution to improve the efficiency of migration and consolidation procedures, by tailoring power saving mechanisms to the specific levels of Quality of Service (QoS) and Quality of Experience (QoE) required by each application.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

¹<http://gigaom.com/2012/03/26/whose-data-centers-are-more-efficient-facebooks-or-googles>.

The development of software for cloud environments is targeting the design of *reactive systems*², which are able to self-adapt to the operational environments and conditions. Enabling development paradigms include meta-models that describe the application structure and its requirements (e.g., TOSCA [7, 8]), and the micro-services architecture [9, 10].

Our idea is to exploit such paradigms in order to drive energy efficiency mechanisms in the data center. Roughly speaking, data center users identify and explicitly mark the VMs that could be run under a low-energy profile, being motivated by lower costs. Consequently, they can pre-provision VMs (for example, replicas for fault-tolerance and scalability purposes), increasing the responsiveness of their applications and maintaining the desired QoS/QoE under highly-varying conditions, while allowing data center operators to achieve very effective energy management.

3. A FRAMEWORK FOR ENERGY EFFICIENCY IN DATA CENTERS

We already proposed a framework for energy efficiency in data centers based on the software development and deployment architecture devised in the ARCADIA project [6].

The ARCADIA project [4] is developing a software architecture that brings autonomicity to cloud management, by exploiting information available in application meta-models, in order to cope with the dynamic and elastic nature of clouds. The architecture includes *i*) a development framework, which assists software and service developers in including context information and policies into their applications, and *ii*) a Smart Controller (SC), which parses the application meta-model and automates most of operations that today are carried out by operation staff (e.g., configuration and deployment) and infrastructure providers (e.g., allocation to meet SLAs, consolidation to accomplish energy-efficient policies) [6]. Further, the Smart Controller also includes monitoring of the current infrastructure, to provide metering information for both application and resource management.

Though the Smart Controller function could be logically embedded within Cloud Management Software (CMS), the current design envisions a separate higher-level element, which allows for deployment over multiple IaaS [11]. In the current implementation plan, the Smart Controller is placed *above* OpenStack and relies on its APIs to manage the cloud resources. This approach is similar to OpenStack Neat [12]. Fig. 1 shows the conceptual architecture of our framework.

For the specific purpose of our work, the SC is responsible for consolidation and energy management. It periodically computes the most efficient VM placement, by taking into account both computing and networking devices, according to the optimization algorithm described in Section 4; it is also responsible to enforce energy management actions, like voltage/frequency scaling and transition to sleep state (for this purpose, it is expected to implement suitable interfaces, like the *Green Abstraction Layer* (GAL) for network equipment [13], which could be extended to cope with computing equipment as well [6].

²The Reactive Manifesto, <http://www.reactivemanifesto.org>

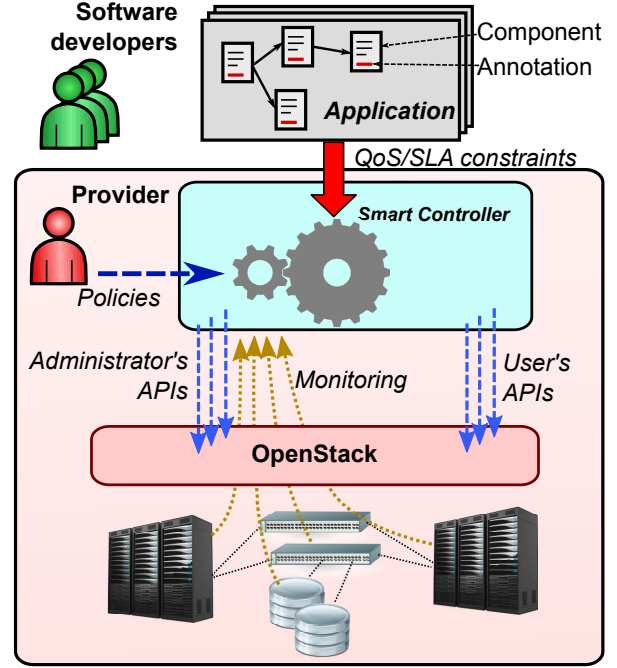


Figure 1: Conceptual architecture of the framework for energy efficiency.

4. ENERGY MANAGEMENT WITH CONTEXT INFORMATION

The novel contribution of this paper is an energy-aware consolidation strategy, exploiting context information annotated by developers and service providers in their applications. We consider micro-services architectures, which split complex applications/services into elementary *software components* that are deployed in dedicated VMs [11]. Each component is classified according to QoS information annotated in the meta-model and their actual performance indexes (percentage of CPU/memory used, service latency, dropped packets/requests, etc.).

Our approach applies different power saving techniques to software components with different QoS constraints and/or workloads, and performs consolidation by keeping into account the same parameters.

4.1 Classification of VMs

In our work, we adopt the following classification:

- *red*, for very critical entities (e.g., they are currently processing medium/high loads, and/or there are tight QoS constraints on availability, processing, bandwidth, latency);
- *yellow*, for busy entities that are not operating close to their performance limit (e.g., they are currently processing low loads, and QoS constraints are largely met);
- *green*, for low-load entities (e.g., they are not currently processing anything or have a very low load, and there are negligible risks of SLA violations).

The above classification is used to understand which VMs could be migrated by a consolidation algorithm and which

VMs could take part into energy management. On the one hand, the fundamental operation for live migration³ is memory copy from the source to the destination, and this operation takes much more bandwidth than necessary when the application is processing data, because memory continuously changes during the copy [14]. On the other hand, power management in some way limits performance (usually it delays the execution due to lower processing power and the time taken to switch hardware components to full operational state). In both cases, we are likely to experience service degradation.

Red entities are critical to move, because any migration takes some time and consumes resources. Hence, migration of red components is likely to turn into QoS/QoE degradation and, ultimately, into SLA violation.

Yellow entities could be grouped together, if they are not using their whole CPU processing power. The SC should not cluster too many of these entities, because there should be enough spare CPU time available to serve sudden peaks of workload. In this case, dynamic voltage/frequency scaling could be used to adapt the performance to the workload, without losing the capability to quickly restore to full operation.

Green entities are not required to be immediately available (i.e., with latency less than a few dozen milliseconds). Green entities will typically be replicas, provisioned to speed up the recovery from failure (maybe of applications in other clouds/data centers) or to respond to requests for scaling up. The SC could cluster these components together, even by aggressive overprovisioning, and can *freeze* them on sleeping servers. Given the above classification, our target is the design of a consolidation strategy that:

- keeps red entities on the smallest number of servers, while assuring enough resources are provisioned to meet their stringent QoS requirements;
- groups yellow components together on the smallest number of servers, even on the same servers as red entities if enough resources are available;
- distributes green components among all servers, with no stringent constraints on grouping, since servers hosting green components only will be put in sleep mode;
- places VMs by considering their traffic patterns, in order to minimize the power consumption of the underlying communication infrastructure.

In the following Sections, we describe the power model and the consolidation algorithm that implement the above outlined strategy.

4.2 Power Model

We consider a multi-component utilization-based power model, which defines the power consumption of a data center according to the resource utilization of server (SMs) and network devices (NMs). The power consumption Φ for modern devices can be realistically modeled as a stepwise function

³Live migration is the movement of one VM from one server to another in real-time. It is only supported in practice when the same hypervisor is used on both servers and when the virtual disk is on a shared network storage. Otherwise, a non-live migration would require to shut down the VM.

of the current utilization U :

$$\Phi = \Phi^0 + U \times \Phi^f \quad (1)$$

where Φ^0 represents the idle power (when no operation is being carried out) and Φ^f is the power consumption at full load. $U \in [0, 1]$ is the current resource utilization; in our case, U corresponds to the fraction of active CPUs and ports for SMs and NMs, respectively.

Obviously, the power consumption for the data center is computed by summing the power consumption of all SMs and NMs.

4.3 Consolidation strategy

The consolidation problem consists in the allocation of VMs on available SMs and in the selection of network paths. The optimization is subjected to constraints on computing/memory resources on SMs and bandwidth on network links. Given the dimension of the problem for realistic environments, the problem is split into two parts, similarly to the approach followed by other researchers [15]: the placement of VMs and then the selection of the routing paths. Once the algorithm has produced the consolidation plan, migration and power management can be applied on both SMs and NMs, according to the framework described in Section 3.

The consolidation happens in two phases: an initial placement of new VMs that enter the system, and a periodic re-placement that considers all VMs in the data center.

The algorithm for initial placement is listed in Algorithm 1. It is executed every time the allocation of a new set of VMs $\mathbb{V}^{(r)}$ enters the system; input parameters are the set of VMs $\mathbb{V}$ already allocated in the SMs and the results of the last optimization ρ (for placement) and φ (for routing).

The algorithm consists of a loop for computing the placement of the new VMs and their routing paths. It starts by updating the list of running VMs calling the `UPDATERUNNINGVM` and copying the list $\mathbb{V}$ to the temporary one $\mathbb{V}^*$ (lines 2–3). The `PLACEMENT` procedure returns the value ρ^* of the optimization function in terms of power consumption and the updated list of VMs (line 5); the placement function tries to group together all VMs, according to the traffic patterns inferred by their bandwidth requirements. If a solution is found ($\rho^* = -1$), the `ROUTING` procedure (line 7) computes the new routing paths trying to minimize the number of requested links and NMs, while guaranteeing the requested bandwidth for each VMs couple. If a solution is found ($\varphi^* \neq -1$), the `ROUTING` function updates the list of VMs; otherwise, the function `DROPREQUEST` removes the VM with high resource requirements from the ones included in the request (lines 8–9). The loop finishes when either a solution is found or all the incoming VMs are dropped. In the first case (lines 11–12), the algorithm updates the list $\mathbb{V}$ and the results of the optimization function ρ and φ .

Since VMs are run for a given time, the allocation might gradually become sub-optimal just as they are de-provisioned, hence necessitating a “re-placement”. The periodic re-placement algorithm is listed in Algorithm 2. It is performed to optimize the allocation of the VMs, taking into consideration the whole set of running VMs.

The re-placement happens by using the same optimization functions of the `INITIALPLACEMENT` algorithm. Migration of VMs is really done only if the results of the optimization functions ρ^* and φ^* are lower than the ones of the last

Algorithm 1 Initial placement

```

1: function INITIALPLACEMENT( $\mathbb{V}, \mathbb{V}^{(r)}, \rho, \varphi$ )
2:    $\mathbb{V} \leftarrow \text{UPDATERUNNINGVM}(\mathbb{V})$ 
3:    $\mathbb{V}^* \leftarrow \mathbb{V}$ 
4:   do
5:      $(\rho^*, \mathbb{V}^*) \leftarrow \text{PLACEMENT}(\mathbb{V}^*, \mathbb{V}^{(r)})$ 
6:     if  $\rho^* \neq -1$  then
7:        $(\varphi^*, \mathbb{V}^*) \leftarrow \text{ROUTING}(\mathbb{V}^*, \mathbb{V}^{(r)})$ 
8:       if  $\rho^* = -1 \vee \varphi^* = -1$  then
9:          $\mathbb{V}^{(r)} \leftarrow \text{DROPREQUEST}(\mathbb{V}^{(r)})$ 
10:    while  $(\rho^* = -1 \vee \varphi^* = -1) \wedge \mathbb{V}^{(r)} \neq \emptyset$ 
11:    if  $\mathbb{V}^{(r)} \neq \emptyset$  then
12:       $(\mathbb{V}, \rho, \varphi) \leftarrow (\mathbb{V}^*, \rho^*, \varphi^*)$ 
13:  return  $(\mathbb{V}, \rho, \varphi)$ 

```

consolidation ρ and φ .

Algorithm 2 Periodic re-placement

```

1: function REPLACEMENT( $\mathbb{V}, \rho, \varphi$ )
2:    $(\rho^*, \mathbb{V}^*) \leftarrow \text{PLACEMENT}(\emptyset, \mathbb{V})$ 
3:   if  $\rho^* < \rho \wedge \rho^* \neq -1$  then
4:      $(\varphi^*, \mathbb{V}^*) \leftarrow \text{ROUTING}(\emptyset, \mathbb{V}^*)$ 
5:     if  $\varphi^* < \varphi \wedge \varphi^* \neq -1$  then
6:        $(\mathbb{V}, \rho, \varphi) \leftarrow (\mathbb{V}^*, \rho^*, \varphi^*)$ 
7:  return  $(\mathbb{V}, \rho, \varphi)$ 

```

4.4 Mathematical formulation

We formulate the placement problem as ILP optimization problem. The placement problem aims at minimizing the total power consumption of SMs guaranteeing the allocation of the resources (in terms of CPUs and RAM) of the incoming requests.

We formulate the routing problem as LP optimization problem. The routing problem aims at minimizing the total power consumption of NMs, while at the same time guaranteeing the communication between VMs belonging to the same application. Specific constraints are used to ensure the positiveness of the variables and that the total bandwidth to allocated is less or equal to the link capacity minus the bandwidth of the VMs already allocated.

For the sake of brevity, we omit the formal problem statement and the detailed mathematical formulation.

5. PERFORMANCE EVALUATION

We wrote an event simulator in MATLAB [16] to validate the proposed algorithm. The simulator implements both the placement and routing algorithms of our energy-aware consolidation strategy and an additional performance-maximization strategy aimed at guaranteeing the highest acceptance rate of requests.

We compare the two procedures to demonstrate the effectiveness of our strategy, by taking into account two performance metrics.

5.1 Simulation Settings

We consider a Data Center Network (DCN) with a three-tier hierarchical topology [17, 18] that has three layers: core, aggregation and edge. In particular, we refer to an architecture using similar k -port switches. There are k PODs (Performance Optimized Data-centers) that contain $k/2$ aggregation and $k/2$ core switches. Instead, the number of

core switches is $k^2/4$ and each core switch has a link to each POD. In this topology, each edge switch is connected to $k/2$ servers. Summarizing, a DCN with k PODs uses $5k^2/4$ port switches and can support $k^3/4$ servers.

In our simulation, we chosen $k = 8$ with a total number of switches and servers equal to 80 and 128, respectively. We used 1 Gbps links between servers and edge switches, 10 Gbps links between edge and aggregation switches, and 100 Gbps between aggregation and core switches. We assumed 10 GB of RAM and 16 CPUs per server.

Table 1 lists the data used for power consumption. We used the same figures as references [19–22]. Idle power of edge, aggregation and core switches is set to 76.4 W, 133.5 W, and 555 W, respectively; additional power for each active port is 0.87 W, 0.9 W and 0.91 W in case of 1 Gbps, 10 Gbps and 100 Gbps links, respectively. Idle power consumption for servers is set to 850 W, while each active CPU consumes 260 W.

Data center traffic characteristics at the packet level between switches has been extensively studied in the literature; traffic distribution such as exponential, weibull, and log-normal have been reported [23, 24]. We consider time-aware requests from tenants, which are generated randomly, where their duration and inter-arrival times are assumed to follow an exponential distribution.

Table 1: Power consumption of the DCN devices.

Device		Power	
Name	Type	Type	Value [W]
Switch	Edge	Static	76.4
	Aggregation		133.5
	Core		555
Port	1 Gbps	Active	0.87
	10 Gbps		0.9
	100 Gbps		0.91
Server		Static	850
		Active	260

5.2 Metrics

We consider the following metrics for performance evaluation:

1. *Acceptance rate* - the average percentage of accepted requests for deploying new VMs. It is computed using the following equation: $\eta_{ac} = (\psi_{VM} - \tilde{\psi}_{VM}) / \psi_{VM}$, where η_{ac} is the acceptance rate, ψ_{VM} and $\tilde{\psi}_{VM}$ are the number of incoming and rejected requests, respectively.
2. *Power savings* - the average power saving for the whole set of NMs and SMs, respectively. This metrics is computed as follows: $\Delta\Phi_{sav} = (\Phi_{max} - \Phi) / \Phi_{max} * 100$, where $\Delta\Phi_{sav}$ is the power saving (in %), Φ_{max} and Φ are the maximum and current power consumption of the device (NM or SM), respectively.

5.3 Scenarios

We consider a simulation time of 1000 units. Unlike other works [22], in this paper, the simulation time domain is con-

tinuous. Hence, a new request can arrive at any time between 0 and 1000. According to the work in [22], we fix the mean arrival rate at 5000 request for time unit and vary the mean request duration between 2 to 12 time unit following a truncated log-normal distribution with mean $\mu = 5$. Each request contains a set of VMs to allocate. The number of VMs included in each request follows a truncated log-normal distribution with mean $\mu = 3$ in the range between 1 and 10.

All the parameters and their statistical properties are listed in Table 2.

5.4 Numerical Results

Using our event simulator, we validated the effectiveness of our energy-aware algorithm, by comparing the results with a straight performance-maximization strategy. The latter does not take into account power consumption, but tries to accommodate the largest number of requests in the data center.

We repeated the experiments for a sufficient number of times to get small errors, representing the 95% confidence interval, as done in [22]. For sake of brevity, we omit additional details about the confidence intervals.

The numerical results are shown in Figs. 2 and 3. Fig. 2 shows that our strategy achieves a power saving between 35% and 45% during the simulation. Instead, the solution that does not take into account the power consumption presents a saving that does not exceed 20%. These results demonstrate the effectiveness of the algorithm proposed for the minimization of power consumption.

Fig. 3 shows very good acceptance rates for both algorithms; the max-performance strategy is never below 98%, and our energy-aware algorithm is always above 92%.

By combining the figures for the two performance indexes, we conclude that our consolidation strategy achieves high power saving with very limited acceptance degradation. In addition, we remark that our consolidation strategy is designed with QoS constraints in mind, so no service degradation is expected on this side.

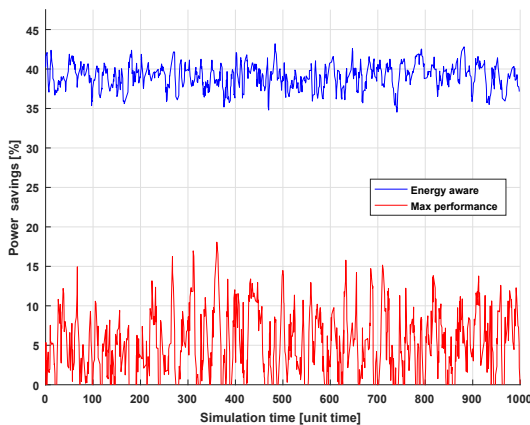


Figure 2: Percentage of the power savings.

6. CONCLUSIONS

In this paper, we have proposed a novel consolidation strategy for energy efficiency. Our work builds on mod-

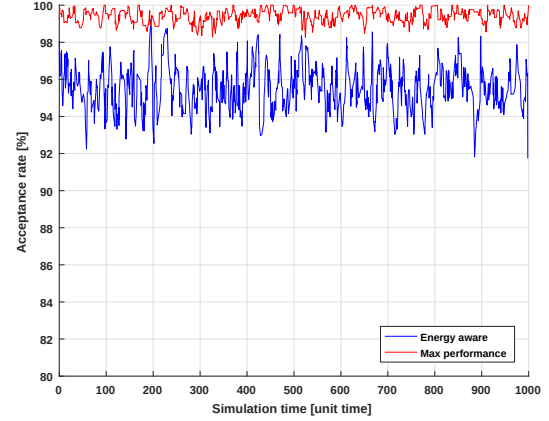


Figure 3: Percentage of the accepted requests.

ern software development paradigms, mainly the usage of meta-data and micro-services architectures.

The main novelty of our framework consists in the ability to save energy while taking into account different QoS constraints for each software component in a modular application.

In this respect, our framework introduces novel dimensions and perspectives for the sustainability of data centers, by enabling an optimal trade-off between the service level expected by application/service providers and energy efficiency pursued by infrastructure providers. We argue that meta-data and annotations could improve sustainability of data centers by considering other aspects in addition to workload consolidation. For example, more specific service level specification and a finer-grained classification of software components and VMs could be used to modulate software execution based on the availability of energy from renewables, and for migration among different data centers whenever needed.

Preliminary simulations show that our consolidation strategy improves the efficiency of the infrastructure, with negligible implications on the acceptance rate of additional service requests. Our next steps will be the evaluation of the impact of the envisioned power saving mechanisms (voltage/frequency scaling, sleep states) in experimental testbeds.

Acknowledgment

This work was supported in part by the European Commission under the project ARCADIA, contract no. 607881.

7. REFERENCES

- [1] “Openstack orchestration – heat,” web site. [Online]. Available: <https://wiki.openstack.org/wiki/Heat>
- [2] “Aws cloudformation documentation,” web site. [Online]. Available: <https://aws.amazon.com/documentation/cloudformation/>
- [3] “Network functions virtualisation (nfv); management and orchestration,” ETSI GS NFV-MAN 001, December 2014, v1.1.1. [Online]. Available: http://www.etsi.org/deliver/etsi_gs/NFV-MAN/001_099/001/01.01.01_60/gs_nfv-man001v010101p.pdf
- [4] [Online]. Available: <http://www.arcadia-framework.eu>
- [5] “Cloudify,” web site. [Online]. Available: <http://getcloudify.org/>

Table 2: Configuration parameters used in the simulation.

Parameter	Distribution	Unit of measurements	Value/ μ	Range [a-b]
Simulation time	Constant	time unit	1000	-
Request arrival rate	Poisson arrival process with Exponential distributed inter-arrival times	number of request/ time unit	5000	-
Request duration	Truncated Log-normal	time unit	5	2–12
Request VM set length	Truncated Log-normal	number of VMs	3	1–10

- [6] R. Bolla, L. Sambolino, D. Tigano, and M. Repetto, “Enhancing energy-efficient cloud management through code annotations and the green abstraction layer,” in *First International Workshop on Sustainable Data Centres and Cloud Computing (SD3C)*, Limassol, Cyprus, Dec. 7, 2015.
- [7] “Topology and orchestration specification for cloud applications,” OASIS Standard, November 2013, version 1.0. [Online]. Available: <http://docs.oasis-open.org/tosca/TOSCA/v1.0/os/TOSCA-v1.0-os.pdf>
- [8] J. Wettinger, U. Breitenbücher, and F. Leymann, “Standards-based DevOps automation and integration using TOSCA,” in *IEEE/ACM 7th International Conference on Utility and Cloud Computing (UCC)*, London, UK, Dec. 8–11, 2014, pp. 59–68.
- [9] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: yesterday, today, and tomorrow,” Cornell University Library, June 2016, arXiv:1606.04036v1. [Online]. Available: <https://arxiv.org/abs/1606.04036v1>
- [10] A. Balalaie, A. Heydarnoori, and P. Jamshidi, “Microservices architecture enables devops: Migration to a cloud-native architecture,” *IEEE Softw.*, vol. 33, no. 3, pp. 42–52, March 2016.
- [11] P. Gouvas and C. V. (Eds), “Description of the ARCADIA framework,” ARCADIA project, Tech. Rep. D2.3, November 2015. [Online]. Available: <http://www.arcadia-framework.eu/wp/wp-content/uploads/2015/12/Arcadia-D2.3-final-otxhv.pdf>
- [12] A. Beloglazov and R. Buyya, “OpenStack Neat: a framework for dynamic and energy-efficient consolidation of virtual machines in OpenStack clouds,” *Concurrency and Computation: Practice and Experience*, vol. 27, no. 5, pp. 1310–1333, April 2015.
- [13] “Green Abstraction Layer (GAL); power management capabilities of the future energy telecommunication fixed network nodes,” ETSI ES 203 237, March 2014, version 1.1.1.
- [14] V. Cima, B. Grazioli, S. Murphy, and T. Bohnert, “Adding energy efficiency to Openstack,” in *Sustainable Internet and ICT for Sustainability (SustainIT)*, Madrid, Spain, Apr. 14–15, 2015, pp. 1–8.
- [15] W. Fang, X. Liang, S. Li, L. Chiaraviglio, and N. Xiong, “VMPlanner: Optimizing virtual machine placement and traffic flow routing to reduce network power costs in cloud data centers,” *Computer Networks*, vol. 57, no. 1, pp. 179–196, January 2013.
- [16] MATLAB, version 9.0.0.341360 (R2016a). Natick, Massachusetts: The MathWorks Inc., 2016.
- [17] M. Al-Fares, A. Loukissas, and A. Vahdat, “A scalable, commodity data center network architecture,” *SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 4, pp. 63–74, Aug. 2008. [Online]. Available: <http://doi.acm.org/10.1145/1402946.1402967>
- [18] R. Niranjan Mysore, A. Pamboris, N. Farrington, N. Huang, P. Miri, S. Radhakrishnan, V. Subramanya, and A. Vahdat, “Portland: A scalable fault-tolerant layer 2 data center network fabric,” *SIGCOMM Comput. Commun. Rev.*, vol. 39, no. 4, pp. 39–50, Aug. 2009. [Online]. Available: <http://doi.acm.org/10.1145/1594977.1592575>
- [19] P. Mahadevan, P. Sharma, S. Banerjee, and P. Ranganathan, “Energy aware network operations,” in *Proceedings of the 28th IEEE International Conference on Computer Communications Workshops*, ser. INFOCOM’09. Piscataway, NJ, USA: IEEE Press, 2009, pp. 25–30. [Online]. Available: <http://dl.acm.org/citation.cfm?id=1719850.1719855>
- [20] B. Heller, S. Seetharaman, P. Mahadevan, Y. Yiakoumis, P. Sharma, S. Banerjee, and N. McKeown, “Elastictree: Saving energy in data center networks,” in *Proceedings of the 7th USENIX Conference on Networked Systems Design and Implementation*, ser. NSDI’10. Berkeley, CA, USA: USENIX Association, 2010, pp. 17–17. [Online]. Available: <http://dl.acm.org/citation.cfm?id=1855711.1855728>
- [21] P. Mahadevan, P. Sharma, S. Banerjee, and P. Ranganathan, “A power benchmarking framework for network devices,” in *Proceedings of the 8th International IFIP-TC 6 Networking Conference*, ser. NETWORKING ’09. Berlin, Heidelberg: Springer-Verlag, 2009, pp. 795–808. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-01399-7_62
- [22] A. Dalvandi, M. Gurusamy, and K. C. Chua, “Time-aware vmflow placement, routing, and migration for power efficiency in data centers,” *IEEE Transactions on Network and Service Management*, vol. 12, no. 3, pp. 349–362, Sept 2015.
- [23] T. Benson, A. Akella, and D. A. Maltz, “Network traffic characteristics of data centers in the wild,” in *Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement*, ser. IMC ’10. New York, NY, USA: ACM, 2010, pp. 267–280. [Online]. Available: <http://doi.acm.org/10.1145/1879141.1879175>
- [24] T. Benson, A. Anand, A. Akella, and M. Zhang, “Understanding data center traffic characteristics,” *SIGCOMM Comput. Commun. Rev.*, vol. 40, no. 1, pp. 92–99, Jan. 2010. [Online]. Available: <http://doi.acm.org/10.1145/1672308.1672325>